Lab 17 — Cloud Shared Responsibility and IAM Least Privilege

> **SY0-701 Objective 3.1** — Compare and contrast security implications of different architecture models
>
> **Domain 3** — Security Architecture (18%) · **Time:** ~25 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: Map who secures each layer in on-premises, IaaS, PaaS and SaaS, then find and fix an over-permissive IAM policy and an insecure infrastructure-as-code template.

You'll build: A responsibility matrix, a small IAM policy simulator, a least-privilege policy, and a policy-as-code scanner that passes a fixed IaC template with zero findings.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; no earlier lab needed; tools installed in step 1. No real cloud account is used — every policy and template is a local, synthetic file.

Background

- The **shared responsibility model** splits security between provider and customer. The further you move from on-premises to IaaS, PaaS and SaaS, the more the provider runs — but the customer **always** keeps data, identities and access settings.
- A **responsibility matrix** documents who owns each control. Most cloud breaches come from customer-side misconfiguration, not provider failure.
- Cloud IAM follows **least privilege** with three evaluation rules: an explicit **Deny** always wins, an explicit **Allow** grants, and anything not mentioned is an **implicit deny**.
- **Infrastructure as code (IaC)** defines servers, networks and storage in templates. One insecure template is copied into every deployment, so templates are scanned before deployment (policy as code).
- Hybrid and third-party arrangements add risk: data crosses environments and every vendor joins your attack surface, while **risk transference** never transfers accountability.

Steps

1. Install the tools

`jq` queries JSON policies and templates, `python3` runs the policy simulator and `column` (from `bsdextrautils`) prints tables.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y jq python3
bsdextrautils
mkdir -p /root/lab17 && cd /root/lab17
jq --version && python3 --version
```

2. Build the shared responsibility matrix

Record who owns each layer in each model as a CSV file — this is your responsibility matrix.

```
cd /root/lab17
cat > matrix.csv <<'EOF'
layer,on_prem,iaas,paas,saas
physical_datacentre,customer,provider,provider,provider
hardware_and_network,customer,provider,provider,provider
hypervisor,customer,provider,provider,provider
guest_os_patching,customer,customer,provider,provider
runtime_and_middleware,customer,customer,provider,provider
application_code,customer,customer,customer,provider
network_rules_security_groups,customer,customer,shared,provider
identity_and_access,customer,customer,customer,customer
data_and_classification,customer,customer,customer,customer
EOF
column -s, -t matrix.csv
```

3. Query the matrix for who owns each control

Answer typical exam questions straight from the matrix.

```
cd /root/lab17
echo "--- IaaS: the customer still owns ---"
awk -F, 'NR>1 && $3=="customer" {print "  " $1}' matrix.csv
echo "--- SaaS: the customer still owns ---"
awk -F, 'NR>1 && $5=="customer" {print "  " $1}' matrix.csv
```

```
--- IaaS: the customer still owns ---
  guest_os_patching
  runtime_and_middleware
  application_code
  network_rules_security_groups
  identity_and_access
  data_and_classification
--- SaaS: the customer still owns ---
  identity_and_access
  data_and_classification
```

4. Create an over-permissive IAM policy

Write an AWS-style policy that grants every action on every resource, then flag wildcard grants with `jq`.

```
cd /root/lab17
cat > policy-v1.json <<'EOF'
{
  "Version": "2012-10-17",
  "Statement": [
    {"Sid": "AdminEverything", "Effect": "Allow", "Action": "*", "Resource": "*"}
  ]
}
EOF
jq -r '.Statement[] | select(.Effect=="Allow")
  | select(([.Action]|flatten|any(.=="*" or endswith(":*"))) or
```

```
([.Resource]|flatten|any(.=="*")))
  | "FINDING: \(.Sid) allows Action=\(.Action) on Resource=\(.Resource)"' policy-v1.json
```

5. Evaluate requests with a policy simulator

This small simulator applies the real cloud IAM logic: explicit deny wins, then allow, otherwise implicit deny.

```
cd /root/lab17
cat > evaluate.py <<'EOF'
#!/usr/bin/env python3
"""Tiny IAM evaluator: explicit Deny > Allow > implicit deny (lab use only)."""
import json, sys, fnmatch
def as_list(v):
    return v if isinstance(v, list) else [v]
policy = json.load(open(sys.argv[1]))
action, resource = sys.argv[2], sys.argv[3]
decision = "IMPLICIT DENY"
for st in policy["Statement"]:
    a_ok = any(fnmatch.fnmatchcase(action, a) for a in as_list(st["Action"]))
    r_ok = any(fnmatch.fnmatchcase(resource, r) for r in as_list(st["Resource"]))
    if a_ok and r_ok:
        if st["Effect"] == "Deny":
            decision = "EXPLICIT DENY"
            break
        decision = "ALLOW"
print(f"{decision:14} {action:24} {resource}")
EOF
cat > requests.txt <<'EOF'
s3:GetObject arn:aws:s3:::reports/q3-summary.pdf
s3:DeleteObject arn:aws:s3:::reports/q3-summary.pdf
iam:CreateUser arn:aws:iam::111122223333:user/backdoor
ec2:TerminateInstances arn:aws:ec2:ap-southeast-1:111122223333:instance/i-0abc123
EOF
while read -r act res; do python3 evaluate.py policy-v1.json "$act" "$res"; done <
requests.txt
```

Every request returns `ALLOW` — including creating a backdoor user. That is the blast radius of a wildcard policy.

6. Rewrite the policy to least privilege and re-evaluate

Allow only reading the `reports` bucket and add an explicit deny on deletion.

```
cd /root/lab17
cat > policy-v2.json <<'EOF'
{
  "Version": "2012-10-17",
  "Statement": [
    {"Sid": "ReadReports", "Effect": "Allow",
     "Action": ["s3:GetObject", "s3:ListBucket"],
     "Resource": ["arn:aws:s3:::reports", "arn:aws:s3:::reports/*"]},
    {"Sid": "NeverDelete", "Effect": "Deny", "Action": "s3:DeleteObject", "Resource": "*"}
  ]
}
EOF
```

```
while read -r act res; do python3 evaluate.py policy-v2.json "$act" "$res"; done <
requests.txt
jq -e '[.Statement[] | select(.Effect=="Allow")
  | select(([.Action]|flatten|any(.=="*" or endswith(":*"))) or
([.Resource]|flatten|any(.=="*")))] | length == 0' \
  policy-v2.json >/dev/null && echo "LEAST PRIVILEGE: no wildcard Allow statements"
```

```
ALLOW          s3:GetObject             arn:aws:s3:::reports/q3-summary.pdf
EXPLICIT DENY  s3:DeleteObject          arn:aws:s3:::reports/q3-summary.pdf
IMPLICIT DENY  iam:CreateUser           arn:aws:iam::111122223333:user/backdoor
IMPLICIT DENY  ec2:TerminateInstances   arn:aws:ec2:ap-southeast-
1:111122223333:instance/i-0abc123
LEAST PRIVILEGE: no wildcard Allow statements
```

7. Scan an IaC template for misconfigurations

Write a simplified IaC template with three classic cloud mistakes, plus a policy-as-code scanner built on `jq`.

```
cd /root/lab17
cat > template-v1.json <<'EOF'
{
  "resources": [
    {"type": "storage_bucket", "name": "reports", "public_read": true, "encryption":
false, "region": "ap-southeast-1"},
    {"type": "security_group", "name": "web-sg",
     "ingress": [{"port": 443, "cidr": "0.0.0.0/0"}, {"port": 22, "cidr": "0.0.0.0/0"}]},
    {"type": "vm", "name": "web01", "image": "ubuntu-24.04", "public_ip": true,
"disk_encrypted": false}
  ]
}
EOF
cat > iac-scan.sh <<'EOF'
#!/usr/bin/env bash
# Minimal policy-as-code checker for the lab template format. Exit 0 = clean.
out=$(jq -r '
  .resources[] as $r
  | ( if $r.type=="storage_bucket" and $r.public_read==true then "HIGH  \($r.name): bucket
is publicly readable" else empty end ),
    ( if $r.type=="storage_bucket" and $r.encryption!=true then "HIGH  \($r.name): bucket
not encrypted at rest" else empty end ),
    ( if $r.type=="security_group" then ($r.ingress[] | select(.cidr=="0.0.0.0/0" and
.port!=443)
        | "HIGH  \($r.name): port \(.port) open to the whole internet") else empty end ),
    ( if $r.type=="vm" and $r.disk_encrypted!=true then "MED   \($r.name): VM disk not
encrypted" else empty end ),
    ( if $r.type=="vm" and $r.public_ip==true then "MED   \($r.name): VM has a public IP
(use a jump server or load balancer)" else empty end )
' "$1")
[ -n "$out" ] && echo "$out"
echo "Findings: $(printf '%s' "$out" | grep -c .)"
[ -z "$out" ]
EOF
chmod +x iac-scan.sh
./iac-scan.sh template-v1.json
```

```
HIGH  reports: bucket is publicly readable
HIGH  reports: bucket not encrypted at rest
HIGH  web-sg: port 22 open to the whole internet
MED   web01: VM disk not encrypted
MED   web01: VM has a public IP (use a jump server or load balancer)
Findings: 5
```

8. Fix the template and confirm a clean scan

Close the bucket, encrypt storage, restrict SSH to the management (jump server) subnet and remove the public IP.

```
cd /root/lab17
cat > template-v2.json <<'EOF'
{
  "resources": [
    {"type": "storage_bucket", "name": "reports", "public_read": false, "encryption":
true, "region": "ap-southeast-1"},
    {"type": "security_group", "name": "web-sg",
     "ingress": [{"port": 443, "cidr": "0.0.0.0/0"}, {"port": 22, "cidr":
"10.0.30.0/24"}]},
    {"type": "vm", "name": "web01", "image": "ubuntu-24.04", "public_ip": false,
"disk_encrypted": true}
  ]
}
EOF
./iac-scan.sh template-v2.json && echo "Template approved for deployment"
diff <(jq -S . template-v1.json) <(jq -S . template-v2.json)
```

Test it

```
cd /root/lab17
./iac-scan.sh template-v2.json
python3 evaluate.py policy-v2.json iam:CreateUser arn:aws:iam::111122223333:user/backdoor
```

Expected: `Findings: 0` (exit code 0) and `IMPLICIT DENY iam:CreateUser ...`. Running `./iac-scan.sh template-v1.json` still reports 5 findings.

Troubleshooting

Symptom	Fix
`jq: error ... any/1 is not defined`	Your jq is very old; run `DEBIAN_FRONTEND=noninteractive apt-get install -y jq` again to get jq 1.6 or later.
`column: command not found`	Install `bsdextrautils`, or read the file with `cat matrix.csv`.
The simulator prints nothing	Check `requests.txt` has two space-separated fields per line and no blank last line with stray spaces.

Challenge

Add a fourth resource type, `database`, with fields `publicly_accessible` and `backup_retention_days`, and extend `iac-scan.sh` to flag public databases and retention below 7 days. Then add a statement to `policy-v2.json` that uses a `Condition` idea of your own (for example, region restriction) and describe how your simulator would need to change.

Reflection

Your company moves its HR system from IaaS virtual machines to a SaaS product. Which rows of your matrix move to the provider, and which rows can never move — and why does that matter for risk transference?

Exam tips

- IaaS: customer patches the guest OS; PaaS: customer secures code and data; SaaS: customer still owns data, identities and access configuration.
- Explicit deny beats any allow; anything not allowed is implicitly denied.
- IaC gives repeatability and removes configuration drift, but one bad template is replicated everywhere — scan templates before deployment.
- Risk transference (to a provider or insurer) never transfers accountability for the data.

References

- NIST SP 800-145 The NIST Definition of Cloud Computing — https://csrc.nist.gov/pubs/sp/800/145/final
- CSA Cloud Controls Matrix — https://cloudsecurityalliance.org/research/cloud-controls-matrix
- AWS Shared Responsibility Model — https://aws.amazon.com/compliance/shared-responsibility-model/
- AWS IAM policy evaluation logic — https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_evaluation-logic.html
- jq manual — https://jqlang.github.io/jq/manual/
